

tesi di laurea magistrale

Studio di Large Language Models a Supporto della Valutazione di Programmi Concorrenti

Anno Accademico 2025/2026

relatori

Ch.mo prof. Roberto Natella

Ch.mo prof. Luigi De Simone

correlatori

Ing. Carmine Cesarano

Ing. Luciano Pianese

candidato

Andrea Esposito

Matr. M63001650

LLM per l'analisi del codice

I modelli linguistici di grandi dimensioni (**LLM**) possono supportare l'analisi del codice software, ma la disponibilità del solo output limita l'individuazione degli errori.

Prompt
esemplificativo



Analizza il seguente codice C, identificando eventuali errori:

```
#include <stdio.h>
int average(int *values, int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {
        total += values[i];
    }
    return total / n;
}
int main() {
    int numbers[] = {};
    int n = sizeof(numbers) / sizeof(numbers[0]);
    printf("Media: %d\n", average(numbers, n));
    return 0;
}
```

+



Problemi principali:

1. **Array vuoto** (int numbers[] = {};) (non valido in C standard).
2. **Possibile divisione per zero** quando n == 0.
3. **Media calcolata come intero**, con perdita della parte decimale.

LLM per l'analisi di programmi concorrenti

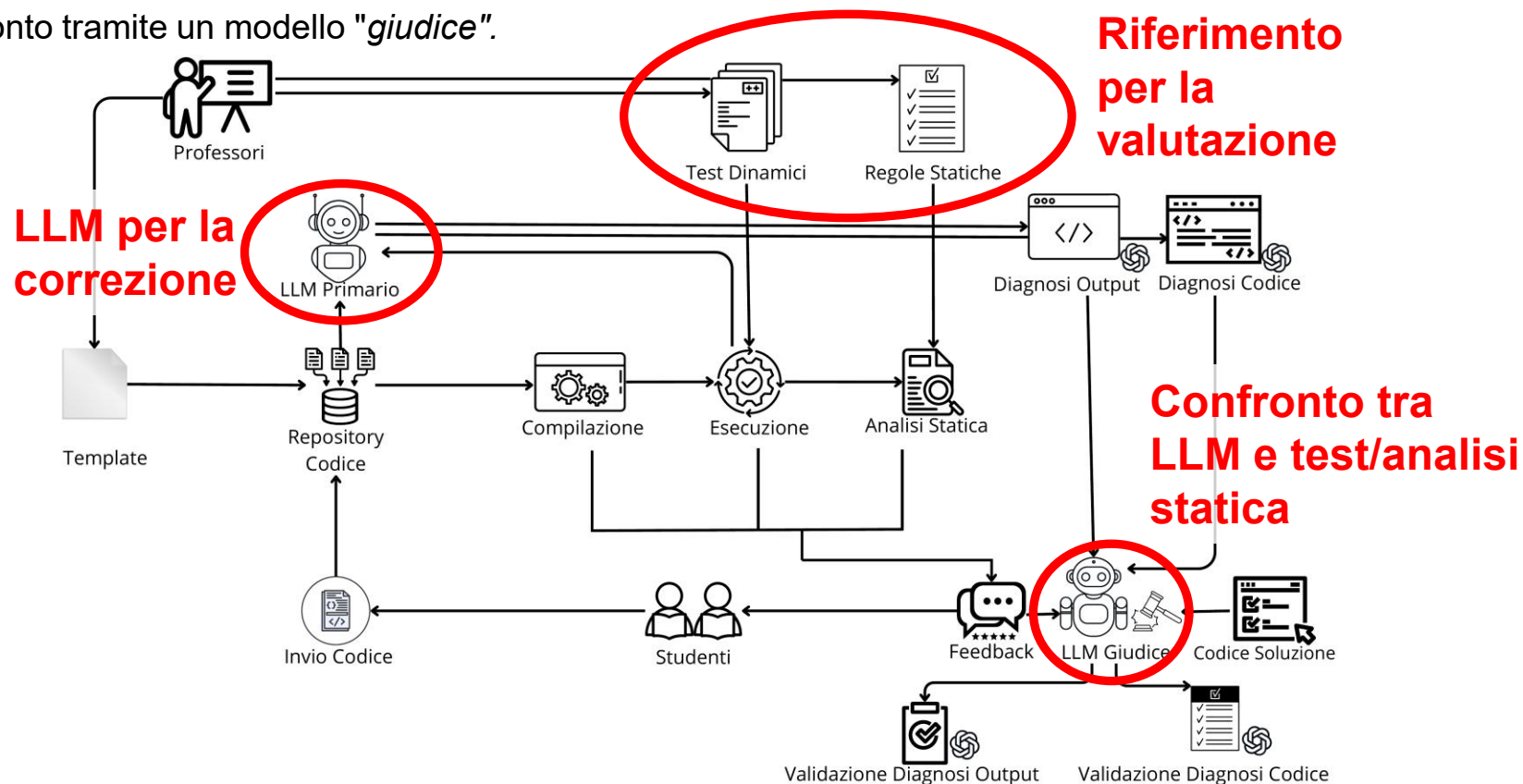
Contributi della tesi:

- Studio sperimentale su uso di LLM per valutare **programmi concorrenti**
- Sviluppo di un **framework di analisi** per la valutazione dei programmi, integrato con il sistema di grading automatico del corso di Sistemi Operativi
- Analisi comparativa rispetto a tecniche classiche di **testing** e **analisi statica**
- **Analisi statica** con ulteriori strumenti e regole

Il campione di programmi analizzati è tratto da **esercitazioni di programmazione** svolte nel contesto del corso di Sistemi Operativi dell'Università di Napoli Federico II

Framework sperimentale

- Il **framework sperimentale** proposto integra testing dinamico (*script bash*), analisi statica (*regole Semgrep personalizzate*), e LLM.
- L'output degli LLM è validato confrontandolo con l'esito dei test e analisi statica.
- Confronto tramite un modello "giudice".



Diagnostica automatica degli esercizi

Esempio di **feedback** prodotto dal sistema di *autograding* integrato nel *framework* proposto, mediante test dinamici e analisi statica



The screenshot displays a dark-themed interface for 'Autograding Feedback'. At the top, it shows 'ghost commented on Nov 26, 2023'. The main title is 'Autograding Feedback'. Below this, it states 'Totale esercizi completati: 1/3'. There are three exercise entries, each with a title and a status message:

- Esercizio struttura dati thread-safe**: Status message: 'L'esercizio non è ancora stato svolto correttamente. Non è stato possibile compilare il programma' (with a warning icon).
- Esercizio lettori-scrittori su più oggetti monitor**: Status message: 'Congratulazioni, l'esercizio compila ed esegue correttamente, e non è stato riscontrato nessuno degli errori frequenti nel codice' (with a success icon).
- Esercizio produttore-consumatore con priorità, con thread**: Status message: 'L'esercizio non è ancora stato svolto correttamente. L'esecuzione non rispetta l'ordine corretto delle produzioni e delle consumazioni' (with a warning icon).

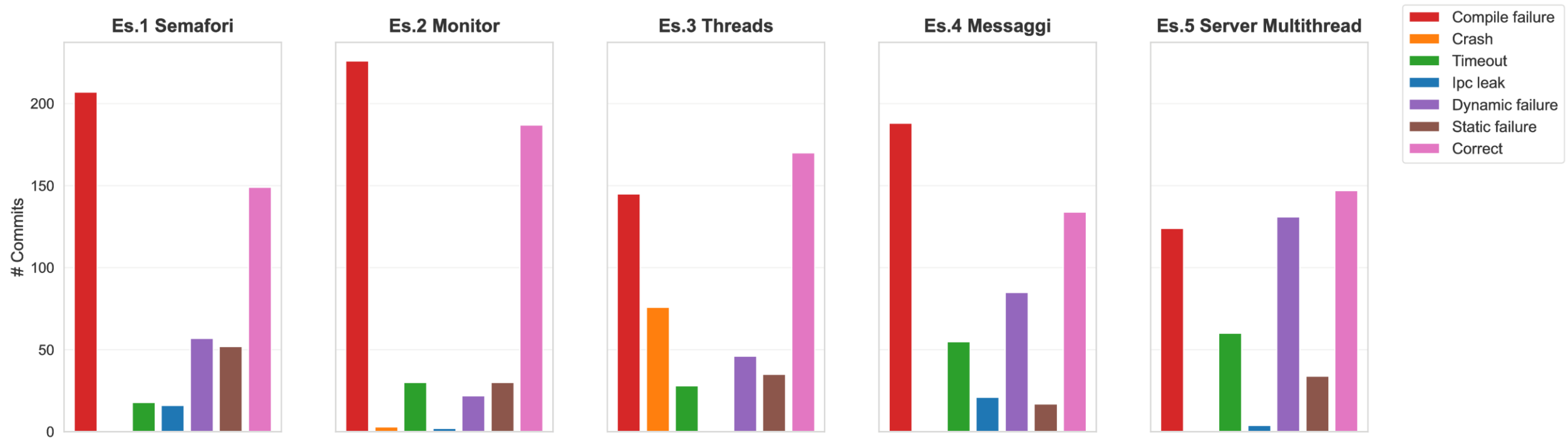
At the bottom left, there is a small circular icon with a smiley face.

Setup sperimentale

- **Workload:** Esercizi di programmazione concorrente svolti dagli studenti, tratti da **5 esercitazioni** del corso di SO
 - Semafori, Monitor, Threads, Scambio Messaggi, Server Multithread
- **Modelli utilizzati:**
 - GPT-4o (cloud deploy su Azure)
 - GPT-OSS-20b (local deploy server DESSERTLAB)
- **Prompt:** progettati attraverso tecniche di *prompt engineering*, con richiesta ai modelli di:
 - **Valutare se l'output** dei programmi degli studenti **è corretto**
 - **Diagnosi dell'output** (se scorretto)
 - **Valutare se il codice** dei programmi **è corretto**
 - **Diagnosi del codice** (se scorretto)
- **Metriche:**
 - **Accuratezza valutazione:** percentuale di classificazioni corrette rispetto a ground truth (esito test dinamici e analisi statica)
 - **Accuratezza della diagnosi:** percentuale di corretta identificazione della causa reale dell'errore (quando presente).

Analisi delle Categorie di Fallimento per ogni Esercizio

Distribuzione delle Categorie di Fallimento per Esercitazione (Analisi con LLM)

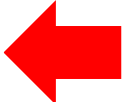


Accuratezza (per esercitazione)

Tipologia di esercitazione	Valutazione Output	Valutazione Codice	Diagnosi Output	Diagnosi Codice
Es.1 Semafori	76.3% (355/465)	96.4% (482/500)	67.9% (38/56)	79.7% (114/143)
Es.2 Monitor	70.3% (327/465)	96.0% (480/500)	47.4% (9/19)	94.3% (82/87)
Es.3 Threads	73.5% (291/396)	90.0% (450/500)	85.0% (34/40)	98.3% (176/179)
Es.4 Messaggi	83.1% (329/396)	97.2% (416/428)	60.0% (39/65)	95.2% (160/168)
Es.5 Server Multithread	81.5% (371/455)	98.8% (486/492)	93.7% (119/127)	94.7% (214/226)

Accuratezza (per categoria di fallimento)

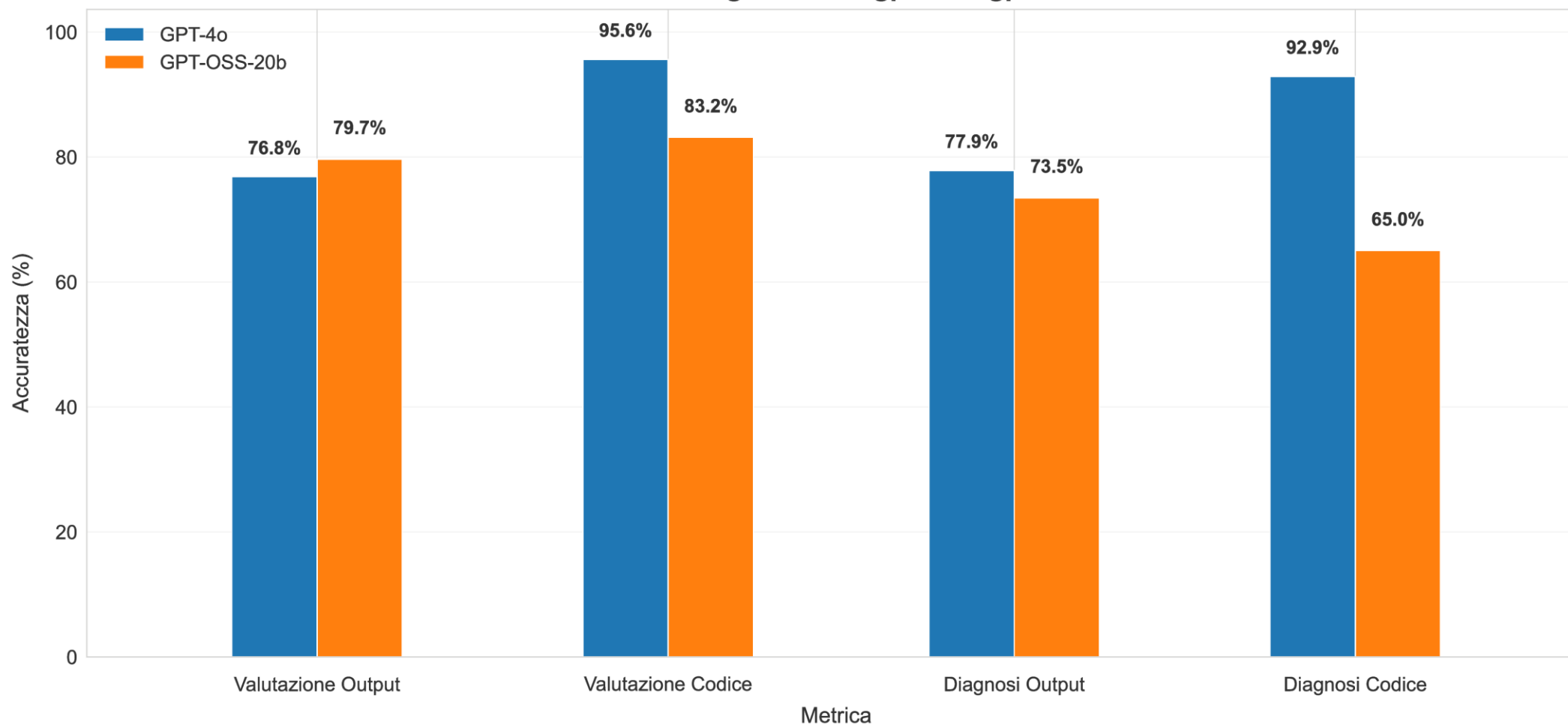
Categoria	Valutazione Output	Valutazione Codice	Diagnosi Output	Diagnosi Codice
crash	N/A	100.0% (80/80)	N/A	100.0% (80/80)
timeout	N/A	100.0% (191/191)	N/A	98.4% (188/191)
ipc_leak	N/A	100.0% (43/43)	N/A	100.0% (43/43)
dynamic_failure	90.0% (307/341)	94.7% (323/341)	77.9% (239/307)	95.4% (308/323)
static_failure	63.7% (107/168)	98.8% (166/168)	100.0% (107/107)	76.5% (127/166)
correct	53.4% (420/787)	89.1% (701/787)	100.0% (420/420)	100.0% (701/701)



I fallimenti trovati dall'analisi statica sono relativi a **problemi di concorrenza** (race condition, deadlock, etc.)

Accuratezza GPT-4o vs GPT-OSS-20b

Confronto globale tra gpt-4o e gpt-oss



Conclusioni

- Lo studio evidenzia buone capacità nel **supporto alla diagnosi degli errori**, soprattutto in presenza di evidenze chiare.
- Nel complesso, gli LLM risultano utili nell'**analisi di programmi concorrenti**, come supporto **complementare** alle tecniche tradizionali di testing e analisi statica.

Grazie per l'attenzione!
Sono a disposizione per eventuali
domande e chiarimenti.